SESSION 3 REFERENCE GUIDE: NUMPY & VECTORIZED NUMERICAL ANALYSIS

This guide provides a reference for NumPy array creation, multi-dimensional slicing, vectorized arithmetic, statistical functions, and axis controls introduced in Session 3.

1. Creating NumPy Arrays & Inspecting Attributes

NumPy arrays hold homogeneous numeric data in contiguous memory.

```
import numpy as np

# Creating 1D Array (Vector)
vector = np.array([10.0, 20.0, 30.0])

# Creating 2D Array (Matrix: 2 rows x 3 columns)
matrix = np.array([
    [100.0, 120.0, 110.0],
    [200.0, 210.0, 195.0]
])

# Inspecting Attributes
print(matrix.shape)  # Output: (2, 3) -> (Rows, Columns)
print(matrix.ndim)   # Output: 2
print(matrix.dtype)  # Output: float64
```

2. Vectorized Operations (No Loops Required)

Vectorization performs element-wise operations instantly across entire arrays.

```
data = np.array([100.0, 200.0, 300.0])

# Scalar Operations
scaled_data = data * 1.05     # Increases all elements by 5%
variance = data - 150.0       # Subtracts 150.0 from every element

# Element-wise Array Operations
array_a = np.array([10.0, 20.0, 30.0])
array_b = np.array([2.0,   4.0,  5.0])
quotient = array_a / array_b  # Result: [5.0, 5.0, 6.0]
```

3. Array Slicing & Indexing

```
matrix = np.array([
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
])

# Syntax: matrix[row_slice, column_slice]
row_0 = matrix[0, :]      # Entire first row: [10, 20, 30]
col_1 = matrix[:, 1]      # Entire second column: [20, 50, 80]
element = matrix[1, 2]    # Specific element at Row 1, Col 2: 60
```

4. Statistical Functions & Axis Control

The `axis` parameter defines the direction of numerical aggregation.

```
grid = np.array([
    [10.0, 20.0],
    [30.0, 40.0]
])

# Overall Matrix Aggregations
total_sum = np.sum(grid)   # 100.0
mean_val  = np.mean(grid)  # 25.0
std_dev   = np.std(grid)   # 11.18

# Directional Aggregations
sum_columns = np.sum(grid, axis=0)  # [40.0, 60.0] -> Down columns
sum_rows    = np.sum(grid, axis=1)  # [30.0, 70.0] -> Across rows
```

5. Boolean Filtering & Masking

```
scores = np.array([85, 42, 90, 55, 61])

# Create Boolean Mask
mask = scores < 60  # [False, True, False, True, False]

# Extract Filtered Sub-array
failing_scores = scores[mask]  # [42, 55]
```

6. The NumPy Analytical Cycle

When analyzing multi-dimensional numerical grids:

[Define Array Matrix] --> [Check .shape] --> [Apply Vectorized Math] --> [Aggregate across Axes] --> [Measure Dispersion (std)]